ZAINA RAHEEN A P

241801328

Learning Objective

The objective of this experiment is to understand and perform Exploratory Data Analysis (EDA) on the Netflix Titles dataset using Python by analysing content types, ratings, countries, release trends, genres, duration, and textual descriptions through statistical methods and visualisations.

Built-in Functions and Methods Used

S.No	Function / Method	Purpose
1	`pd.read_csv()`	Reads the Netflix CSV dataset and converts it into a DataFrame.
2	`df.head()`	Displays the first few rows of the dataset.
3	`df.shape`	Returns the number of rows and columns in the dataset.
4	`df.info()`	Displays column names, data types, and non-null values.
5	`df.describe()`	Provides statistical summary of numerical and categorical data.
6	`df.isnull().sum()`	Counts missing values in each column.
7	`df.duplicated().sum()`	Identifies the number of duplicate rows.
8	`value_counts()`	Counts the occurrences of unique values in a column.
9	`dropna()`	Removes missing values from selected data.
10	`str.split()`	Splits strings such as countries, genres, and actors into individual values.
11	`explode()`	Converts lists in a column into separate rows.
12	`pd.to_datetime()`	Converts the `date_added` column into date/time format.
13	`dt.year`	Extracts the year from a date.
14	`dt.month_name()`	Extracts the month name from a date.
15	`sort_values()`	Sorts the dataset based on a column such as release year.
16	`groupby()`	Groups data based on a particular column for analysis.
17	`pd.crosstab()`	Creates a frequency table between two categorical variables.
18	`str.extract()`	Extracts numerical values such as movie duration or number of seasons.
19	`astype()`	Converts data from one data type to another.
20	`plt.figure()`	Creates a figure for plotting graphs.
21	`plot()`	Creates line, bar, and other statistical plots.
22	`plt.hist()`	Displays the distribution of movie durations.
23	`plt.show()`	Displays the generated graph.
24	`sns.heatmap()`	Creates a heatmap to show genre co-occurrence.
25	`WordCloud()`	Generates a word cloud from Netflix descriptions.
26	`Counter()`	Counts the frequency of words or other items.

```
from google.colab import files
uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from collections import Counter
from wordcloud import WordCloud
```

```
df = pd.read_csv("netflix_titles.csv")
```


https://colab.research.google.com/drive/1WnaSwQ136hRgeQ4qIez-5WvFQPH4YQgC#scrollTo=0WipHGN-2nbm&printMode=true 2/16

```
df.head()
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	description
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States	September 25, 2021	2020	PG-13	90 min	Documentaries	As her father nears the end of his life, filmm...
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Thaban...	South Africa	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, TV Dramas, TV Mysteries	After crossing paths at a party, a Cape Town t...
					Sami							

```
print("Shape:", df.shape)
```

```
Shape: (8807, 12)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype 
---  ------        --------------  ----- 
 0   show_id       8807 non-null   object
 1   type          8807 non-null   object
 2   title         8807 non-null   object
 3   director      6173 non-null   object
 4   cast          7982 non-null   object
 5   country       7976 non-null   object
 6   date_added    8797 non-null   object
 7   release_year  8807 non-null   int64 
 8   rating        8803 non-null   object
 9   duration      8804 non-null   object
 10  listed_in     8807 non-null   object
 11  description   8807 non-null   object
dtypes: int64(1), object(11)
memory usage: 825.8+ KB
```

```
df.describe(include="all")
```

	show_id	type	title	director	cast	country	date_added	release_year	rating	duration	listed_in	descripti
count	8807	8807	8807	6173	7982	7976	8797	8807.000000	8803	8804	8807	88
unique	8807	2	8807	4528	7692	748	1767	NaN	17	220	514	87
top	s8807	Movie	Zubaan	Rajiv Chilaka	David Attenborough	United States	January 1, 2020	NaN	TV-MA	1 Season	Dramas, International Movies	Paranorm activity a lus abandon prope
freq	1	6131	1	19	19	2818	109	NaN	3207	1793	362	
mean	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2014.180198	NaN	NaN	NaN	Na
std	NaN	NaN	NaN	NaN	NaN	NaN	NaN	8.819312	NaN	NaN	NaN	Na
min	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1925.000000	NaN	NaN	NaN	Na
25%	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2013.000000	NaN	NaN	NaN	Na
50%	NaN	NaN	NaN	NaN	NaN	NaN	NaN	2017.000000	NaN	NaN	NaN	Na

```
df.isnull().sum()
```

	0
show_id	0
type	0
title	0
director	2634
cast	825
country	831
date_added	10
release_year	0
rating	4
duration	3
listed_in	0
description	0

dtype: int64

```
df.duplicated().sum()
```

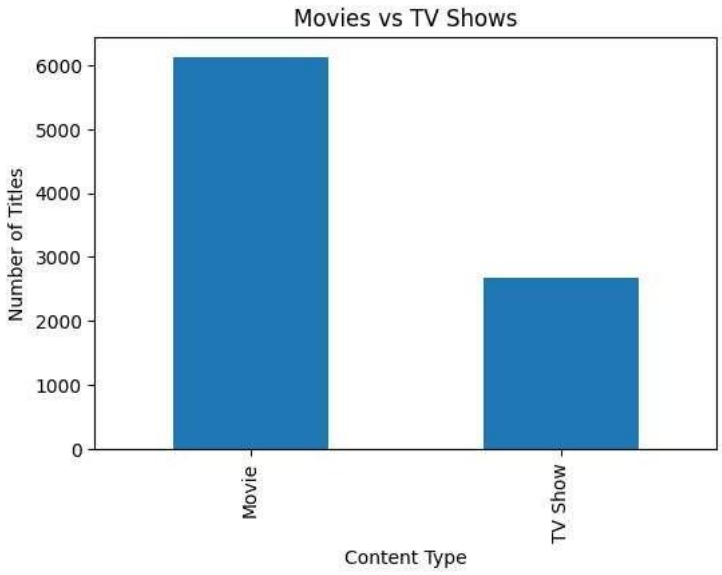
```
np.int64(0)
```

```
type_count = df['type'].value_counts()

print(type_count)

plt.figure(figsize=(6,4))
type_count.plot(kind='bar')
plt.title("Movies vs TV Shows")
plt.xlabel("Content Type")
plt.ylabel("Number of Titles")
plt.show()
```

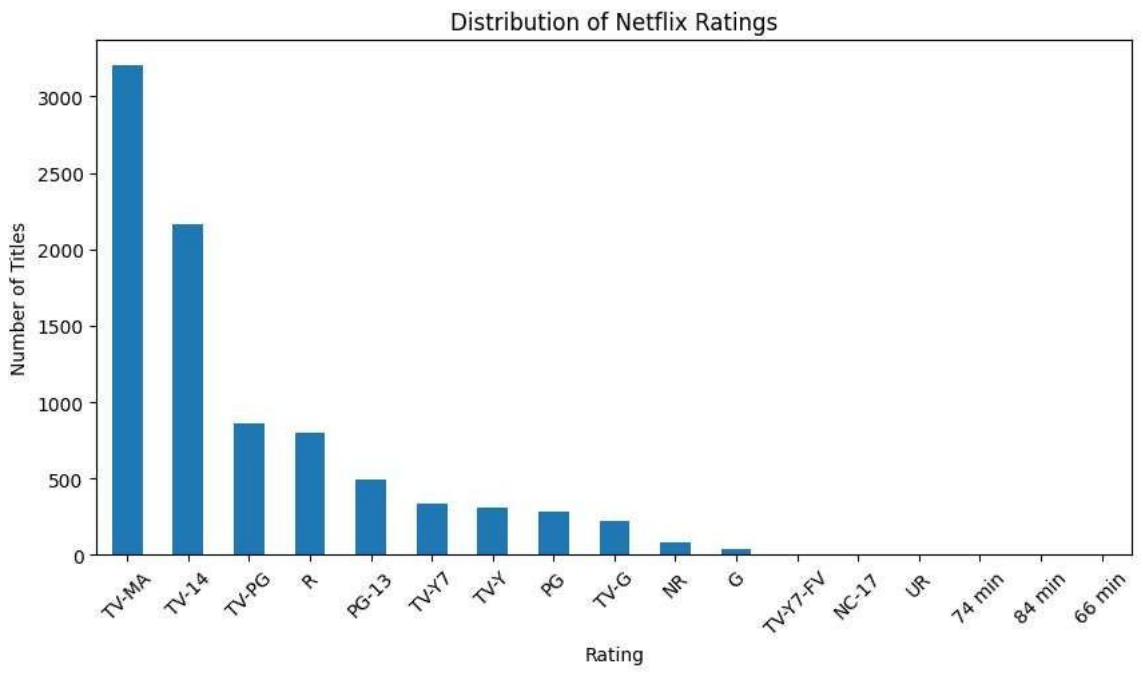
```
type
Movie      6131
TV Show    2676
Name: count, dtype: int64
```




```
rating_count = df['rating'].value_counts()

plt.figure(figsize=(10,5))
rating_count.plot(kind='bar')
plt.title("Distribution of Netflix Ratings")
plt.xlabel("Rating")
plt.ylabel("Number of Titles")
```

```
plt.xticks(rotation=45)
plt.show()
```




```
print(rating_count)
```

```
rating
TV-MA       3207
TV-14       2160
TV-PG        863
R            799
PG-13        490
TV-Y7        334
TV-Y         307
PG           287
TV-G         220
NR            80
G             41
TV-Y7-FV       6
NC-17          3
UR             3
74 min         1
84 min         1
66 min         1
Name: count, dtype: int64
```

```
countries = df['country'].dropna().str.split(', ').explode()

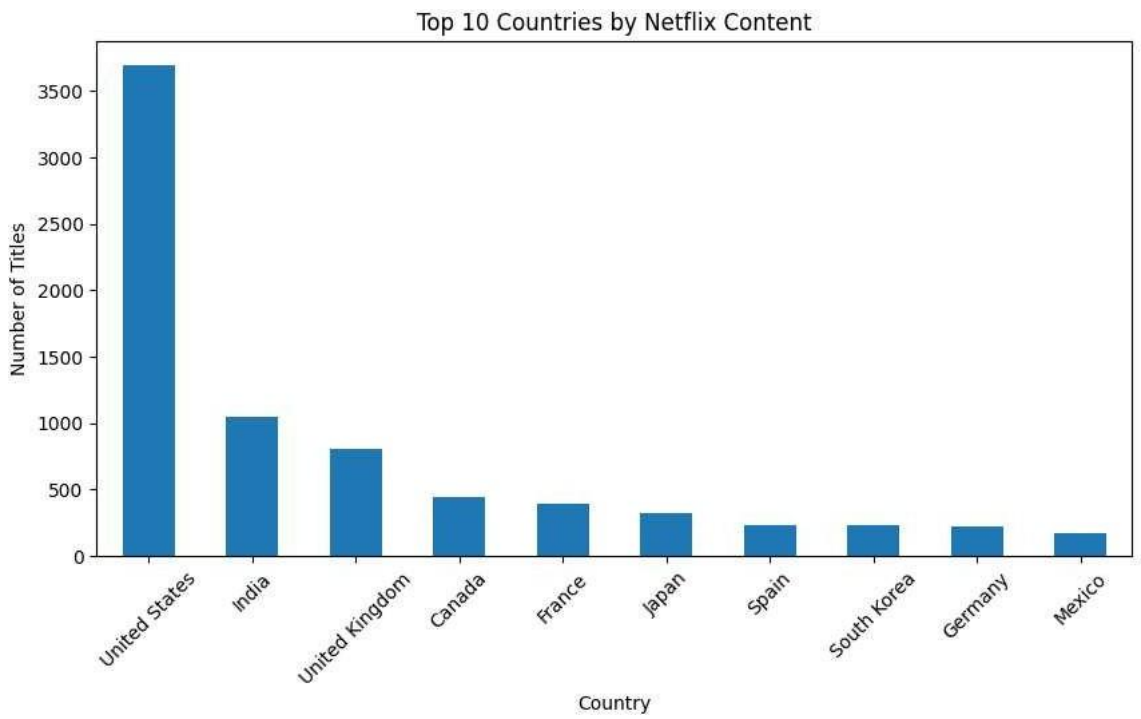
country_count = countries.value_counts().head(10)

print(country_count)
```

```
country
United States     3689
India             1046
United Kingdom     804
Canada             445
France             393
Japan              318
Spain              232
South Korea        231
Germany            226
Mexico             169
Name: count, dtype: int64
```

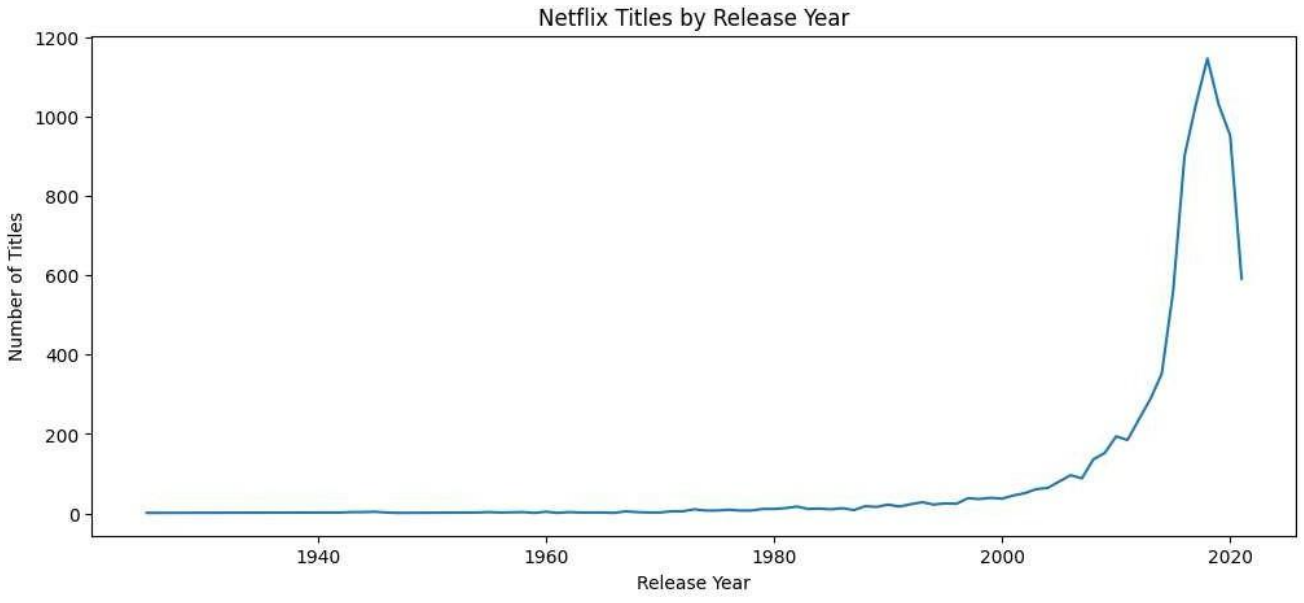
```
plt.figure(figsize=(10,5))
country_count.plot(kind='bar')
plt.title("Top 10 Countries by Netflix Content")
plt.xlabel("Country")
plt.ylabel("Number of Titles")
```

```
plt.xticks(rotation=45)
plt.show()
```




```
release_count = df['release_year'].value_counts().sort_index()

plt.figure(figsize=(12,5))
release_count.plot()
plt.title("Netflix Titles by Release Year")
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")
plt.show()
```




```
print(df['release_year'].value_counts().head(10))
```

```
release_year
2018    1147
2017    1032
2019    1030
2020     953
2016     902
2021     592
```

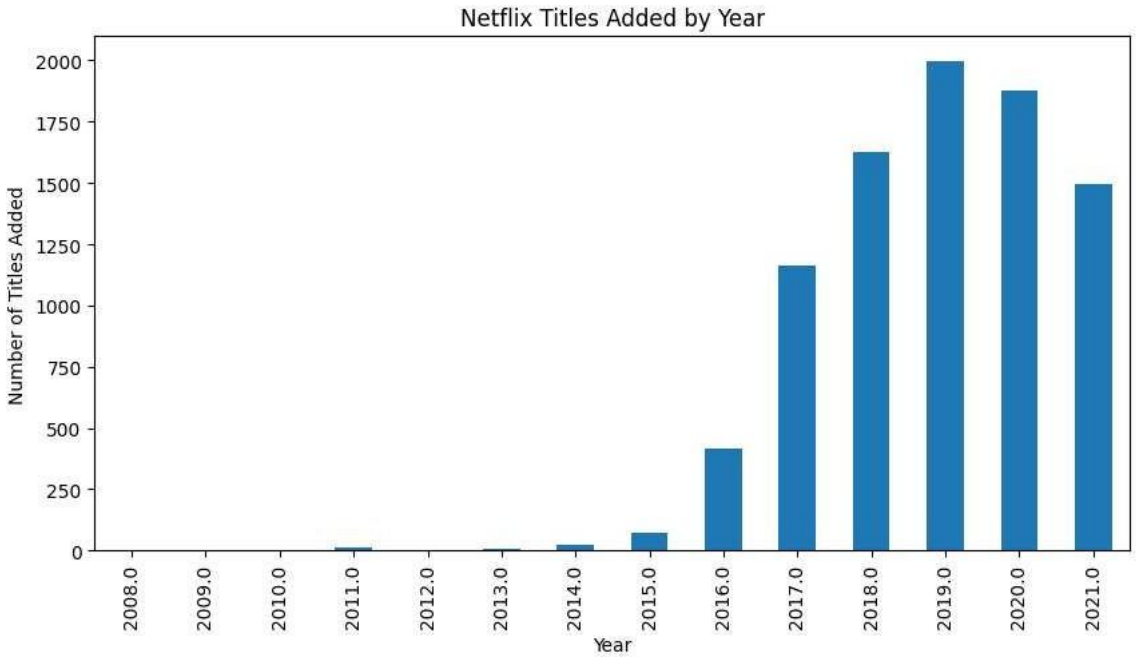
```
2015     560
2014     352
2013     288
2012     237
Name: count, dtype: int64
```

```
df['date_added'] = pd.to_datetime(df['date_added'], errors='coerce')
```

```
df['added_year'] = df['date_added'].dt.year
```

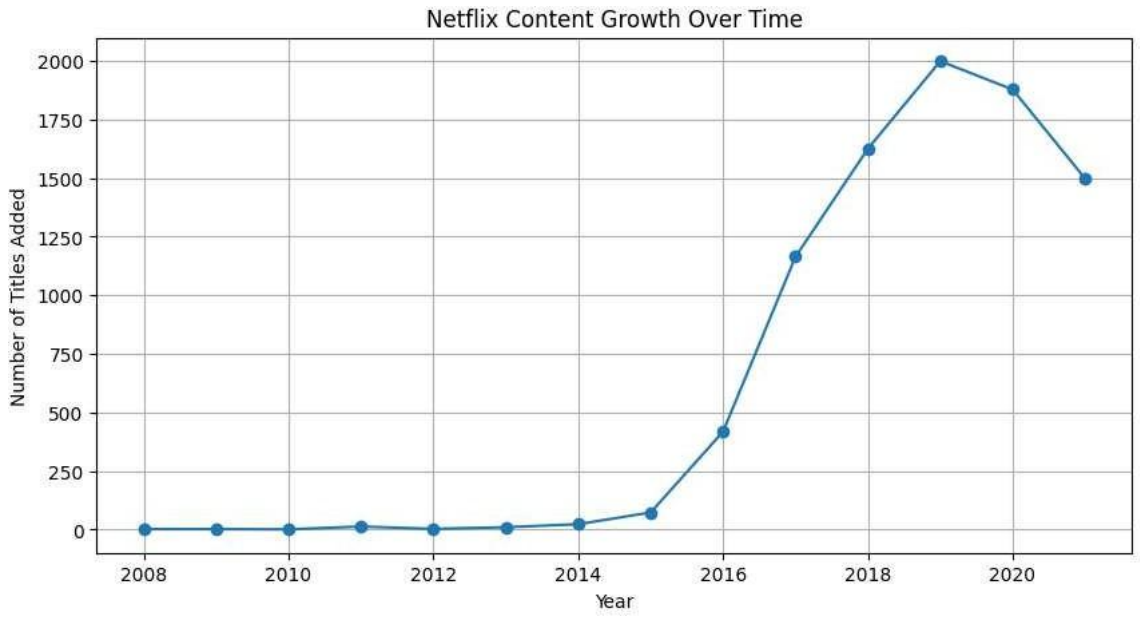
```
added_year_count = df['added_year'].value_counts().sort_index()

plt.figure(figsize=(10,5))
added_year_count.plot(kind='bar')
plt.title("Netflix Titles Added by Year")
plt.xlabel("Year")
plt.ylabel("Number of Titles Added")
plt.show()
```




```
growth = df.groupby('added_year').size()

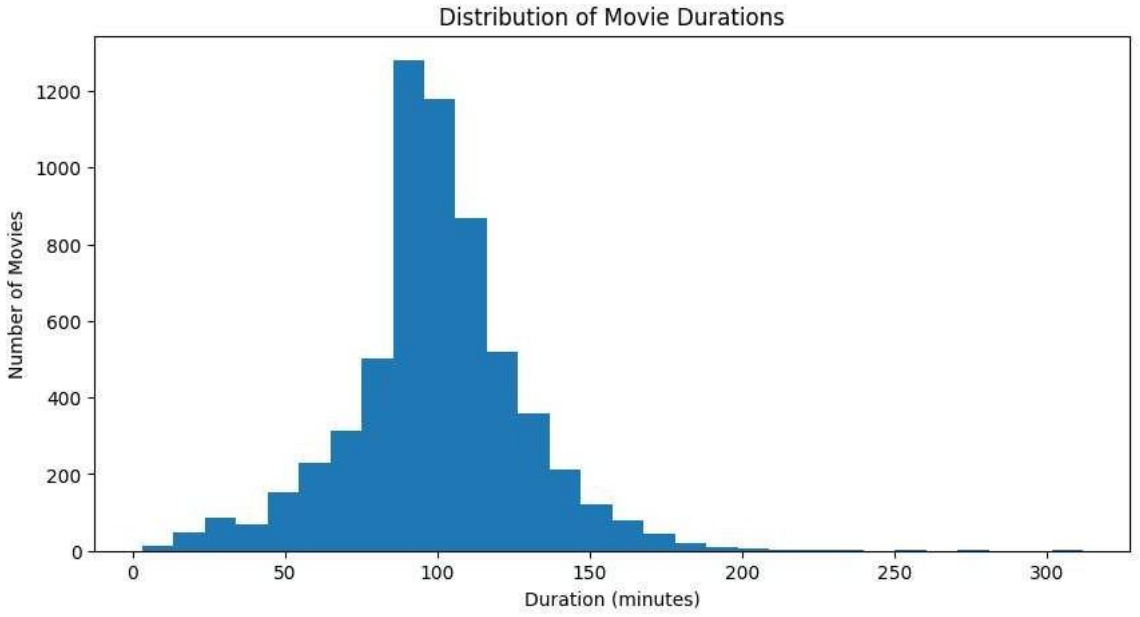
plt.figure(figsize=(10,5))
plt.plot(growth.index, growth.values, marker='o')
plt.title("Netflix Content Growth Over Time")
plt.xlabel("Year")
plt.ylabel("Number of Titles Added")
plt.grid()
plt.show()
```




```
movies = df[df['type'] == 'Movie'].copy()

movies['duration_minutes'] = (
    movies['duration']
    .str.extract(r'(\d+)')[0]
    .astype(float)
)
```

```
plt.figure(figsize=(10,5))
plt.hist(movies['duration_minutes'].dropna(), bins=30)
plt.title("Distribution of Movie Durations")
plt.xlabel("Duration (minutes)")
plt.ylabel("Number of Movies")
plt.show()
```




```
tv = df[df['type'] == 'TV Show'].copy()

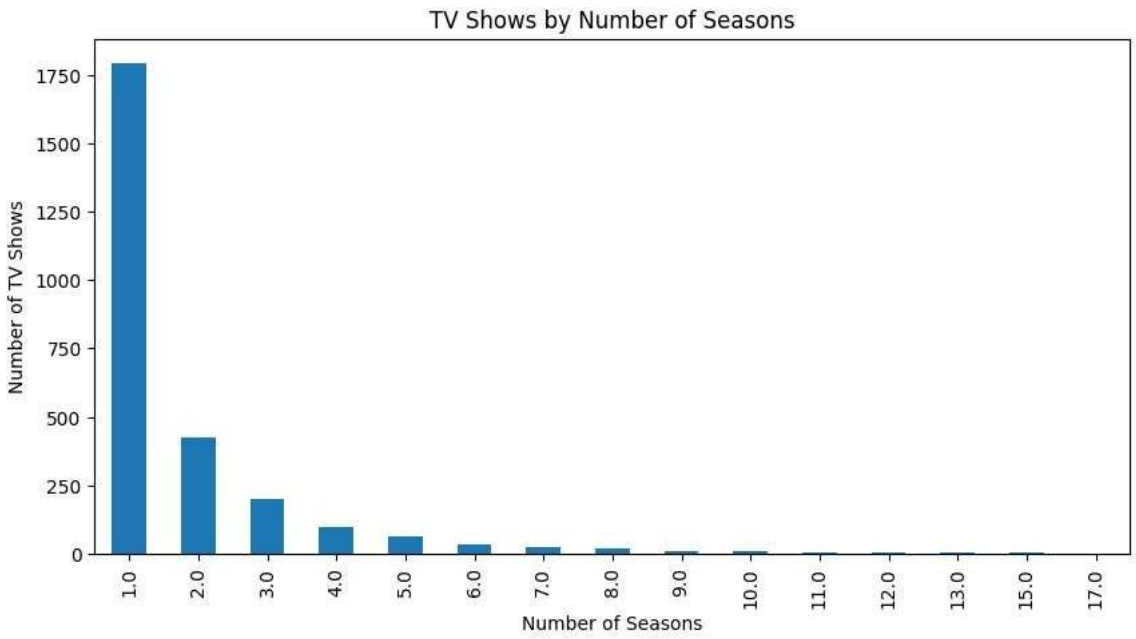
tv['seasons'] = (
    tv['duration']
    .str.extract(r'(\d+)')[0]
    .astype(float)
)
```

```
season_count = tv['seasons'].value_counts().sort_index()

print(season_count)
```

```
seasons
1.0     1793
2.0      425
3.0      199
4.0       95
5.0       65
6.0       33
7.0       23
8.0       17
9.0        9
10.0       7
11.0       2
12.0       2
13.0       3
15.0       2
17.0       1
Name: count, dtype: int64
```

```
plt.figure(figsize=(10,5))
season_count.plot(kind='bar')
plt.title("TV Shows by Number of Seasons")
plt.xlabel("Number of Seasons")
plt.ylabel("Number of TV Shows")
plt.show()
```




```
genres = df['listed_in'].dropna().str.split(', ').explode()

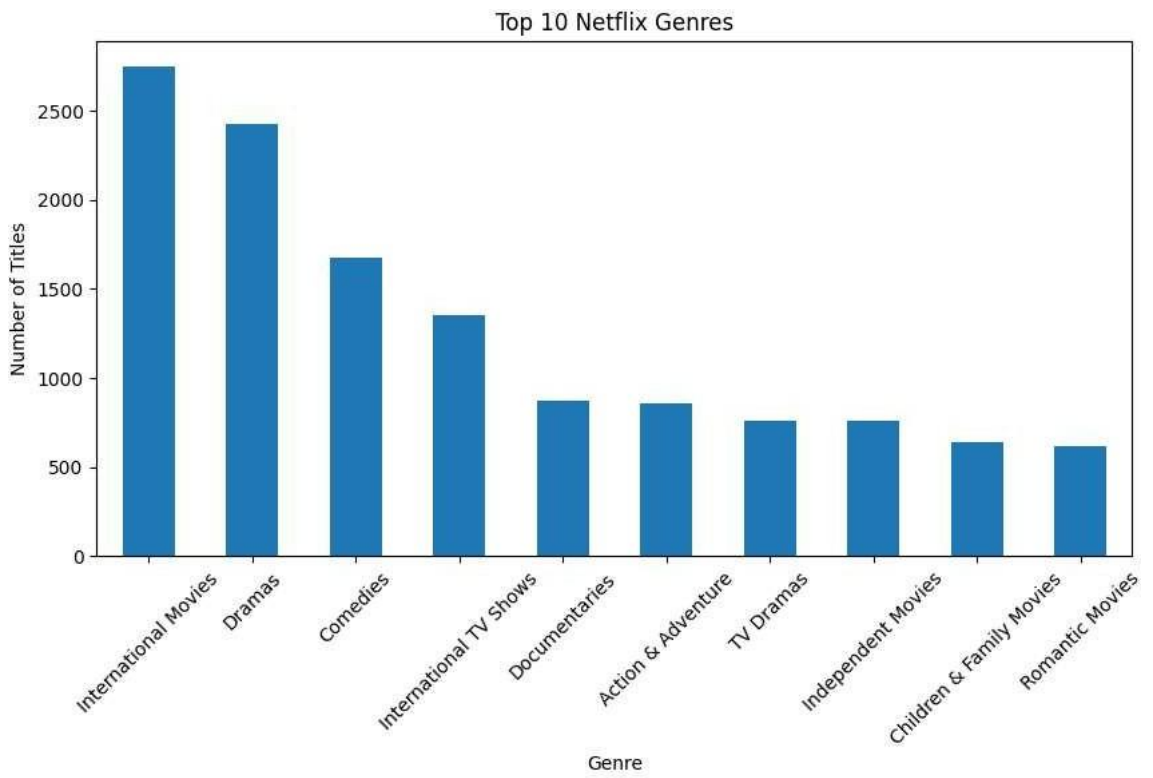
top_genres = genres.value_counts().head(10)

print(top_genres)
```

```
listed_in
International Movies        2752
Dramas                      2427
Comedies                    1674
International TV Shows      1351
Documentaries                869
Action & Adventure           859
TV Dramas                    763
Independent Movies           756
Children & Family Movies     641
Romantic Movies              616
Name: count, dtype: int64
```

```
plt.figure(figsize=(10,5))
top_genres.plot(kind='bar')
plt.title("Top 10 Netflix Genres")
```

```
plt.xlabel("Genre")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
```



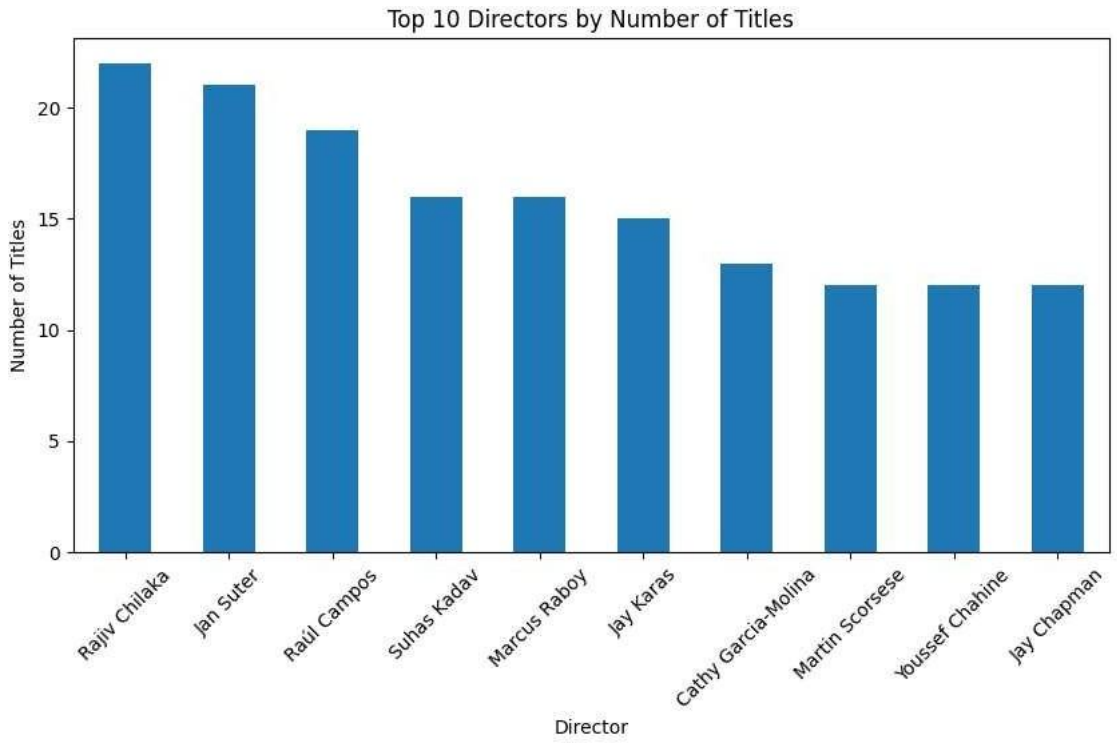

```
directors = df['director'].dropna().str.split(', ').explode()

top_directors = directors.value_counts().head(10)

print(top_directors)
```

```
director
Rajiv Chilaka          22
Jan Suter              21
Raúl Campos            19
Suhas Kadav            16
Marcus Raboy           16
Jay Karas              15
Cathy Garcia-Molina    13
Martin Scorsese        12
Youssef Chahine        12
Jay Chapman            12
Name: count, dtype: int64
```

```
plt.figure(figsize=(10,5))
top_directors.plot(kind='bar')
plt.title("Top 10 Directors by Number of Titles")
plt.xlabel("Director")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
```



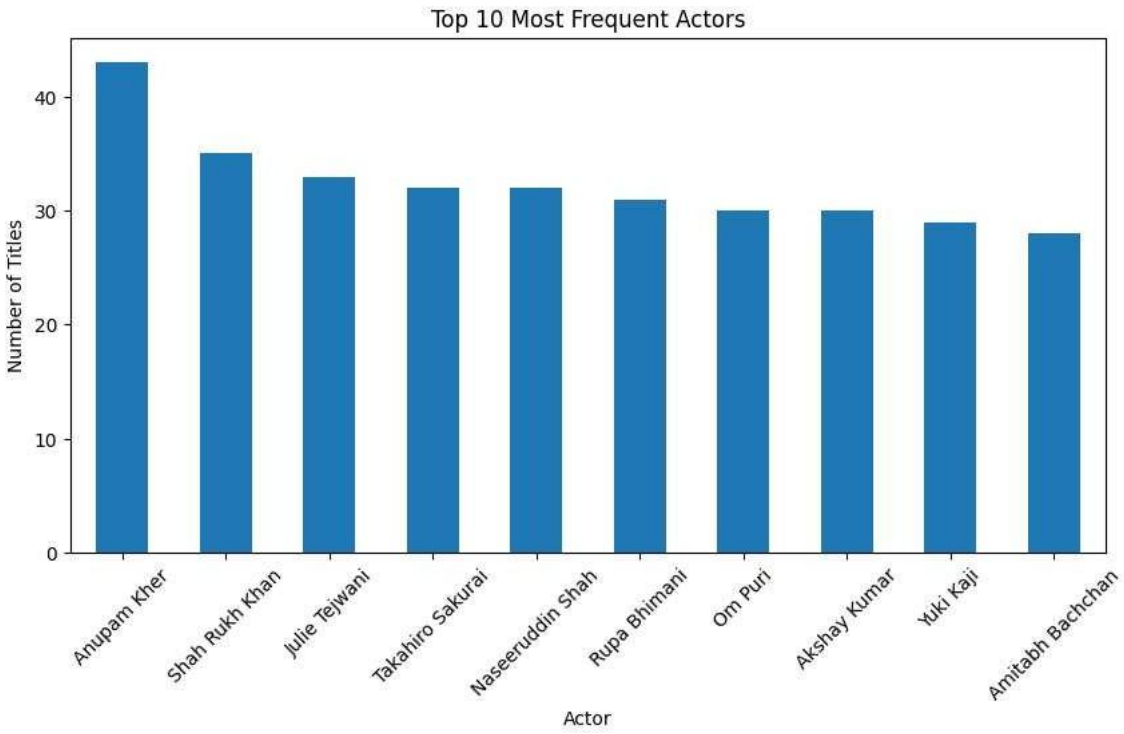

```
actors = df['cast'].dropna().str.split(', ').explode()

top_actors = actors.value_counts().head(10)

print(top_actors)
```

```
cast
Anupam Kher          43
Shah Rukh Khan       35
Julie Tejwani        33
Takahiro Sakurai     32
Naseeruddin Shah     32
Rupa Bhimani         31
Om Puri              30
Akshay Kumar         30
Yuki Kaji            29
Amitabh Bachchan     28
Name: count, dtype: int64
```

```
plt.figure(figsize=(10,5))
top_actors.plot(kind='bar')
plt.title("Top 10 Most Frequent Actors")
plt.xlabel("Actor")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
```




```
text = " ".join(df['description'].dropna())
```

```
wordcloud = WordCloud(
    width=1000,
    height=500,
    background_color='white'
).generate(text)
```

```
plt.figure(figsize=(12,6))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title("Most Common Words in Netflix Descriptions")
plt.show()
```




```
df['added_month'] = df['date_added'].dt.month_name()
```

```
month_count = df['added_month'].value_counts()

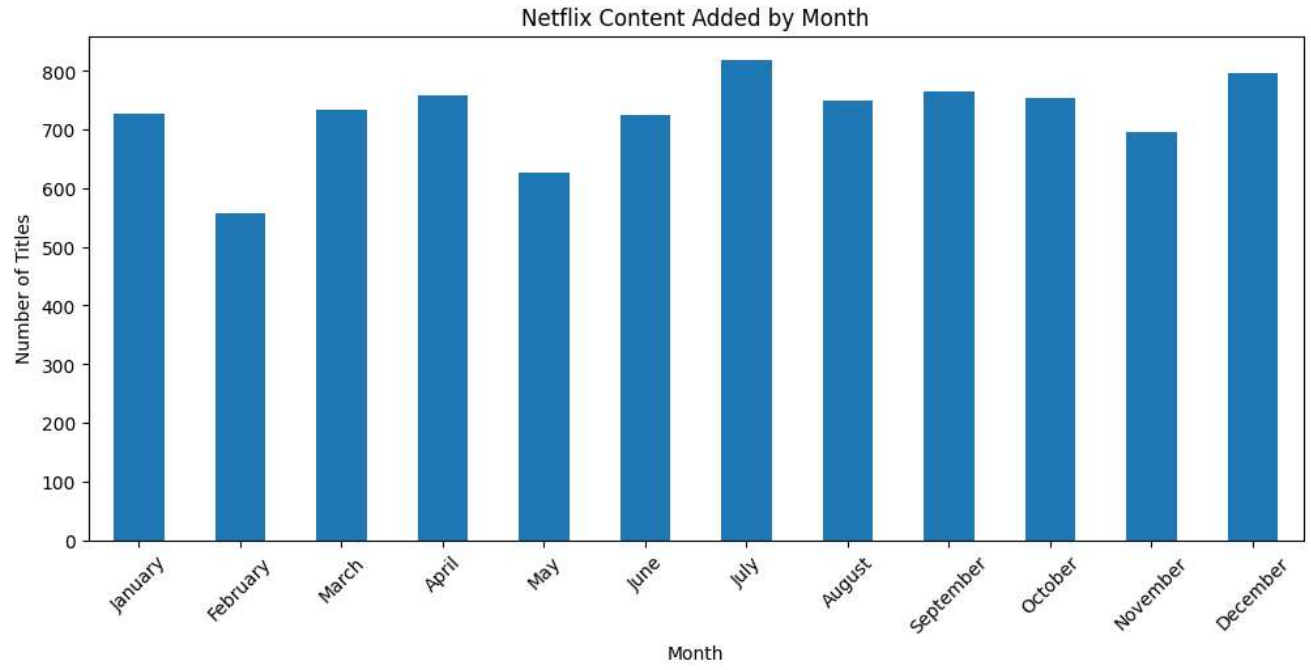
print(month_count)
```

```
added_month
July         819
December     797
September    765
April        759
October      755
August       749
March        734
January      727
June         724
November     697
May          626
February     557
Name: count, dtype: int64
```

```
months = [
    'January', 'February', 'March', 'April',
    'May', 'June', 'July', 'August',
    'September', 'October', 'November', 'December'
]

month_count = df['added_month'].value_counts().reindex(months)
```

```
plt.figure(figsize=(12,5))
month_count.plot(kind='bar')
plt.title("Netflix Content Added by Month")
plt.xlabel("Month")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
```




```
country_type = df[['country', 'type']].dropna().copy()

country_type['country'] = country_type['country'].str.split(', ')
country_type = country_type.explode('country')

top_countries = country_type['country'].value_counts().head(10).index

country_type = country_type[
    country_type['country'].isin(top_countries)
]
```

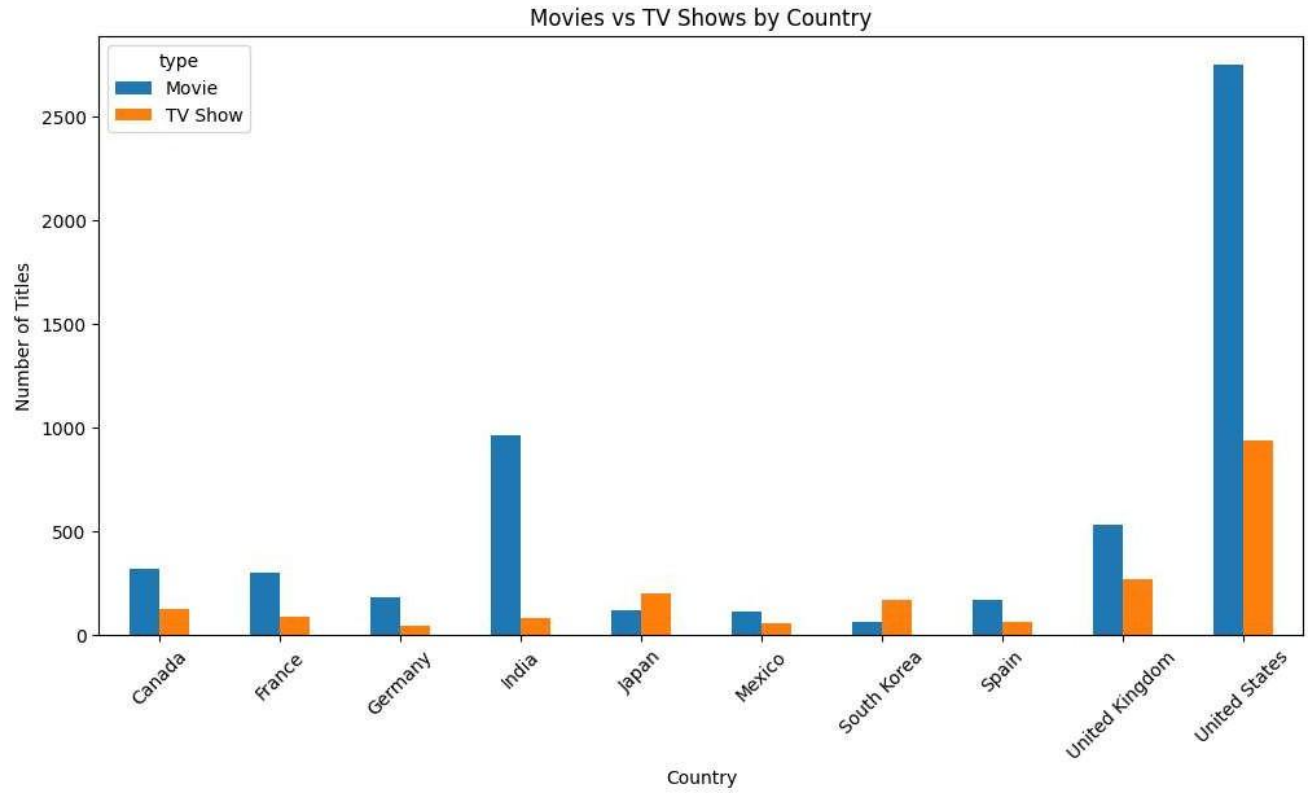
```
comparison = pd.crosstab(
    country_type['country'],
    country_type['type']
)

print(comparison)
```

```
type            Movie  TV Show
country
Canada            319      126
France            303       90
Germany           182       44
India             962       84
Japan             119      199
Mexico            111       58
South Korea        61      170
Spain             171       61
United Kingdom    532      272
United States    2751      938
```

```
comparison.plot(
    kind='bar',
    figsize=(12,6)
)

plt.title("Movies vs TV Shows by Country")
plt.xlabel("Country")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
```




```
rating_type = pd.crosstab(
    df['rating'],
    df['type']
)

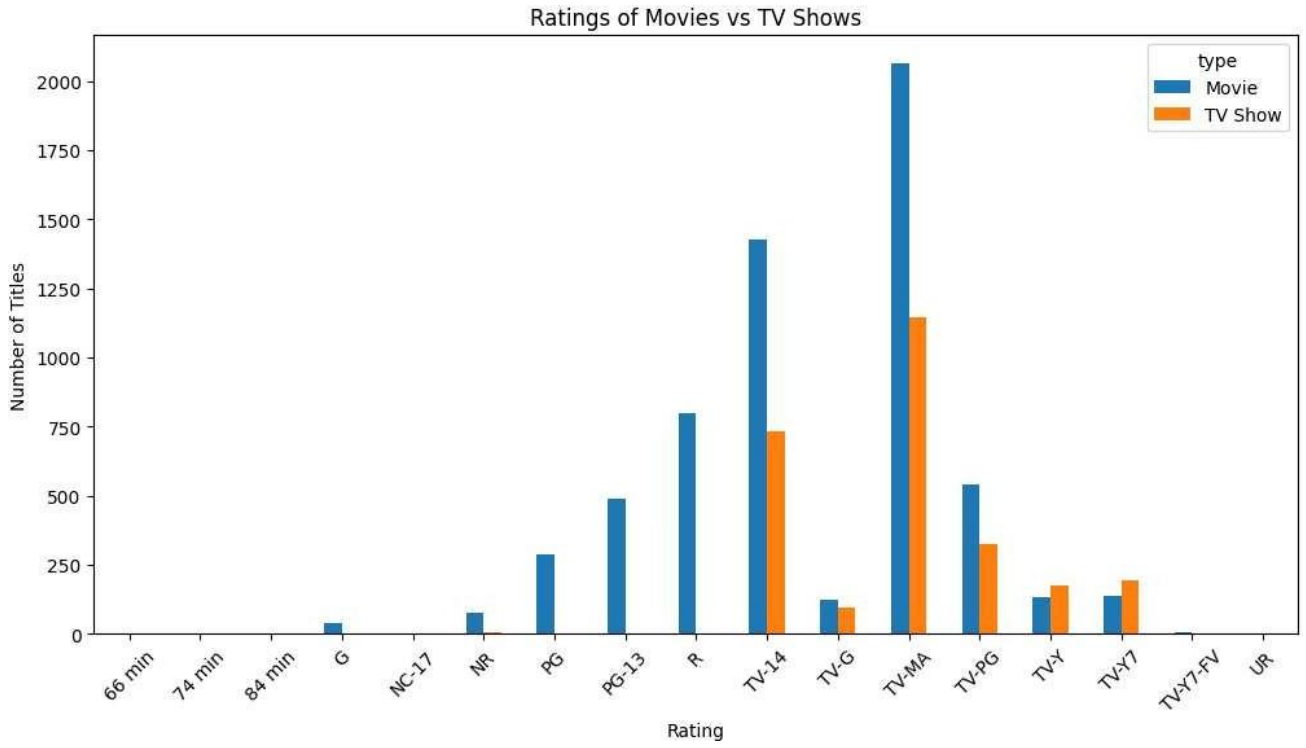
print(rating_type)
```

```
type      Movie  TV Show
rating
66 min        1        0
74 min        1        0
84 min        1        0
G            41        0
```

```
NC-17         3        0
NR           75        5
PG          287        0
PG-13       490        0
R           797        2
TV-14      1427      733
TV-G        126       94
TV-MA      2062     1145
TV-PG       540      323
TV-Y        131      176
TV-Y7       139      195
TV-Y7-FV      5        1
UR            3        0
```

```
rating_type.plot(
    kind='bar',
    figsize=(12,6)
)

plt.title("Ratings of Movies vs TV Shows")
plt.xlabel("Rating")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
```




```
oldest = df.sort_values('release_year').head(10)

print(oldest[['title', 'type', 'release_year']])
```

```
                                          title     type  release_year
4250                Pioneers: First Women Filmmakers*  TV Show          1925
7790                                  Prelude to War    Movie          1942
8205                              The Battle of Midway    Movie          1942
8739                Why We Fight: The Battle of Russia    Movie          1943
8763                   WWII: Report from the Aleutians    Movie          1943
8660     Undercover: How to Operate Behind Enemy Lines    Movie          1943
8419  The Memphis Belle: A Story of a\nFlying Fortress    Movie          1944
8436                                The Negro Soldier    Movie          1944
8640                                 Tunisian Victory    Movie          1944
7219                           Know Your Enemy - Japan    Movie          1945
```

```
newest = df.sort_values('release_year', ascending=False).head(10)

print(newest[['title', 'type', 'release_year']])
```

```
                                          title     type  release_year
1                                 Blood & Water  TV Show          2021
2                                     Ganglands  TV Show          2021
3                             Jailbirds New Orleans  TV Show          2021
31                               Chicago Party Aunt  TV Show          2021
30                                   Ankahi Kahaniya    Movie          2021
25                            Love on the Spectrum  TV Show          2021
23    Go! Go! Cory Carson: Chrissy Takes the Wheel    Movie          2021
20  Monsters Inside: The 24 Faces of Billy Milligan  TV Show          2021
19                                        Jaguar  TV Show          2021
18                                     Intrusion    Movie          2021
```

```
genre_lists = df['listed_in'].dropna().str.split(', ')

all_genres = sorted(set(
    genre
    for genres in genre_lists
    for genre in genres
))
```

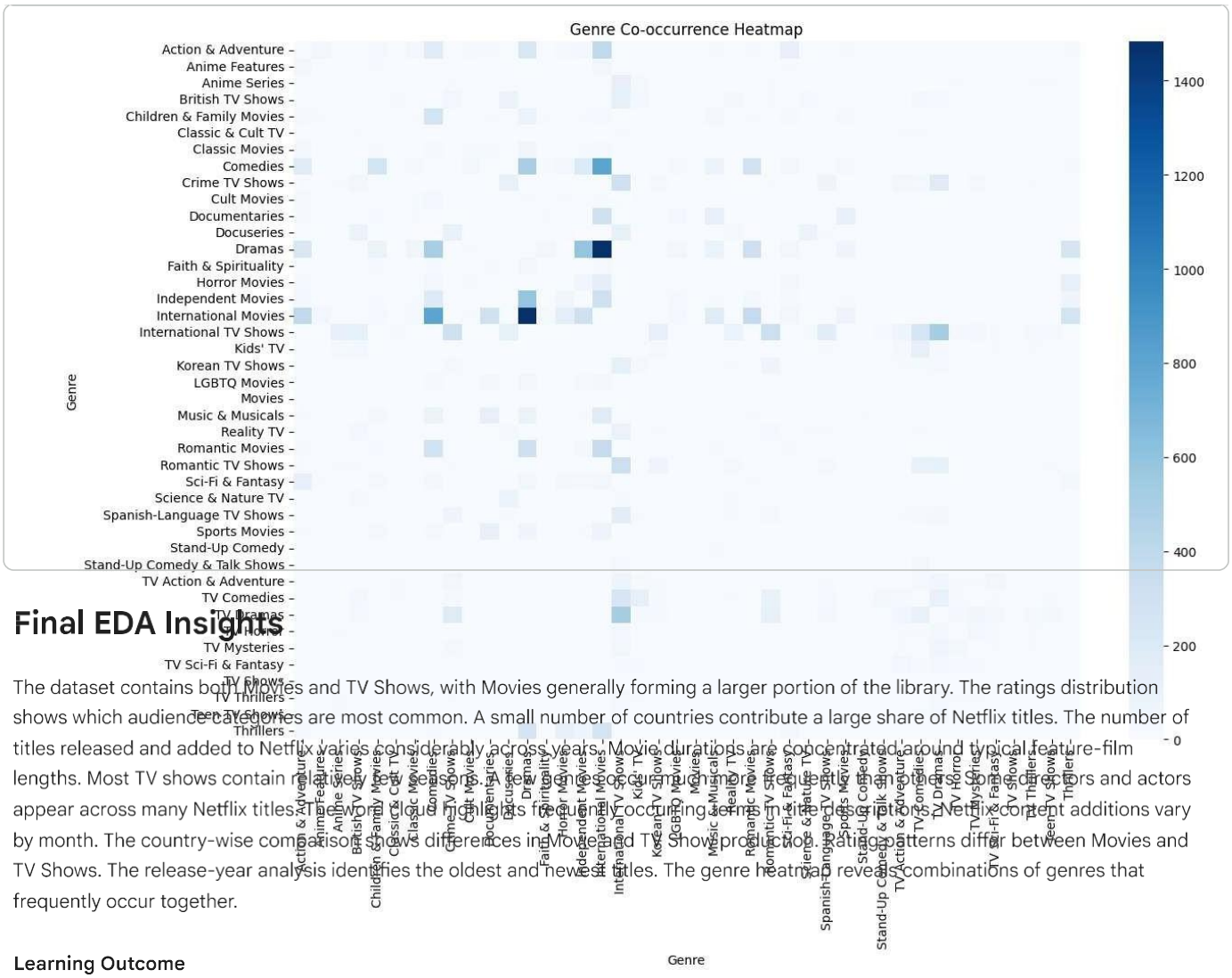
```
genre_matrix = pd.DataFrame(
    0,
    index=all_genres,
    columns=all_genres
)
```

```
for genres in genre_lists:
    for genre1 in genres:
        for genre2 in genres:
            if genre1 != genre2:
                genre_matrix.loc[genre1, genre2] += 1
```

```
plt.figure(figsize=(14,10))

sns.heatmap(
    genre_matrix,
    cmap='Blues'
)

plt.title("Genre Co-occurrence Heatmap")
plt.xlabel("Genre")
plt.ylabel("Genre")
plt.show()
```



Final EDA Insights

The dataset contains both Movies and TV Shows, with Movies generally forming a larger portion of the library. The ratings distribution shows which audience categories are most common. A small number of countries contribute a large share of Netflix titles. The number of titles released and added to Netflix varies considerably across years. Movie durations are concentrated around typical feature-film lengths. Most TV shows contain relatively few seasons. A few genres occur much more frequently than others. Some directors and actors appear across many Netflix titles. The word cloud highlights frequently occurring terms in title descriptions. Netflix content additions vary by month. The country-wise comparison shows differences in Movie and TV Show production. Rating patterns differ between Movies and TV Shows. The release-year analysis identifies the oldest and newest titles. The genre heatmap reveals combinations of genres that frequently occur together.

Learning Outcome

I was able to perform Exploratory Data Analysis on the Netflix dataset using Python, identify patterns and trends in the content, and interpret the results using suitable statistical methods and visualisations.

Double-click (or enter) to edit